



南開大學
Nankai University

计算机学院
语音信息处理实验报告

实验四：MeloTTS-Homework 实验报
告

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2026 年 6 月 9 日

目录

1 课程一：MeloTTS 的安装（15%） 2

2 课程二：多音字语音合成（30%） 3

3 课程三：韵律控制（45%） 5

4 课程四：MeloTTS 的训练与推理优化（加分项 10%） 9

4.1 所选选题：选题一 9

4.2 实现过程 9

4.3 评测结果 12

1 课程一：MeloTTS 的安装（15%）

问题一 MeloTTS 的 Python API (melo/api.py) 中, `tts_to_file` 方法有以下参数: `text`, `speaker_id`, `output_path`, `sdp_ratio`, `noise_scale`, `noise_scale_w`, `speed`, `pbar`, `format`, `position`, `quiet`。请解释每个参数的含义及作用，可结合 `melo/api.py` 源码和实际修改参数后的合成效果进行说明。

答 `tts_to_file` 是 MeloTTS 对外提供的核心语音合成接口。它首先根据语言对输入文本分句，然后对每个句子进行文本前端处理，得到 `phones`、`tones`、`lang_ids`、`bert`、`ja_bert` 等模型输入；随后调用模型的 `infer` 函数生成每句语音，最后把多段音频拼接，并在句间加入短静音。如果指定了输出路径，就写为音频文件；否则返回 `numpy` 音频数组。其函数签名如下：

```
1 def tts_to_file(  
2     self, text, speaker_id, output_path=None,  
3     sdp_ratio=0.2, noise_scale=0.6, noise_scale_w=0.8,  
4     speed=1.0, pbar=None, format=None, position=None, quiet=False  
5 ):  
6     ...  
7
```

各参数含义如下：

1. **text**: 待合成的输入文本，是 TTS 的源输入。源码中会先调用 `split_sentences_into_pieces` 分句，再进行文本正则化、G2P、BERT 特征提取等处理。文本中的标点、数字、英文、多音字都会影响最终音素序列和停顿。
2. **speaker_id**: 说话人编号，用于多说话人模型中选择具体音色。它会被转换成 `LongTensor` 并传入模型。修改该参数时，文本内容不变，但说话人的音色、口音和部分韵律习惯会改变。
3. **output_path**: 输出文件路径。若为字符串，则用 `soundfile.write` 保存音频；若为 `None`，则不写文件而直接返回合成出的音频数组。
4. **sdp_ratio**: 控制随机时长预测器 `Stochastic Duration Predictor` 与确定性时长预测结果的混合比例。该值较小时，音素时长更稳定，语音更可控；较大时，韵律变化更明显，但也可能造成节奏不稳。
5. **noise_scale**: 控制声学隐变量采样时的噪声幅度。较小会使语音稳定、保守，较大则增加细节和随机性，但过大时可能出现颤动、发音不清或音色异常。
6. **noise_scale_w**: 控制时长预测相关噪声的尺度，主要影响音素持续时间、停顿和节奏。它与 `sdp_ratio` 都影响韵律，但 `sdp_ratio` 控制随机时长预测器参与比例，`noise_scale_w` 控制采样噪声强度。
7. **speed**: 整体语速控制参数。在源码中它被转换为 `length_scale=1./speed`。因此 `speed=1.0` 为正常语速，`speed>1` 语速变快，`speed<1` 语速变慢。句间静音长度也会随 `speed` 缩放。
8. **pbar**: 自定义进度条包装函数，只影响多句文本推理时的进度显示，不影响合成音频。
9. **format**: 指定输出音频封装格式，如 `WAV`、`FLAC`。若为空，则由 `soundfile` 根据文件后缀推断格式。

10. **position**: 传给 `tqdm` 的进度条位置参数，用于多进度条显示场景，不影响语音内容。
11. **quiet**: 控制是否减少运行输出。为 `True` 时不显示分句信息和普通进度条；为 `False` 时会打印分句结果并显示进度。

实验中对 `speed` 的修改最直观：当 `speed=0.75` 时，合成语音整体变慢，句间停顿也相应变长；当 `speed=1.5` 时，语音整体变快。对 `sdp_ratio` 和 `noise_scale_w` 的修改则主要影响局部音长和韵律随机性；对 `noise_scale` 的修改主要影响音色细节和稳定性。

2 课程二：多音字语音合成（30%）

问题一 请阅读 `melo/text` 下的 `chinese.py`、`english.py`、`chinese_mix.py` 三个脚本，介绍一下 MeloTTS 是如何分别对中文、英文和中英混合文本进行正则化预处理的？

答 MeloTTS 的文本前端会先进行文本正则化，将原始文本中不适合直接送入 G2P 的形式转换为规范文本。不同语言的规则不同：中文侧重数字转中文读法、中文标点统一和非法字符过滤；英文侧重大小写归一、时间/数字/缩写展开；中英混合则既保留中文又保留英文，并在 G2P 阶段分别送入对应语言模块。

中文脚本 `chinese.py` 的正则化入口为：

```
1 def text_normalize(text):
2     numbers = re.findall(r"\d+(?:\.\d+)?", text)
3     for number in numbers:
4         text = text.replace(number, cn2an.an2cn(number), 1)
5     text = replace_punctuation(text)
6     return text
7
```

其中 `cn2an.an2cn` 将阿拉伯数字转为中文读法，例如 123 转为中文数字表达，避免数字字符在中文 G2P 中无法处理。随后 `replace_punctuation` 会把中文标点映射为模型支持的标点，例如顿号、逗号、分号映射为 `,`，句号映射为 `.`，问号映射为 `?`，感叹号映射为 `!`。最后正则过滤掉非中文汉字和非允许标点的字符，因此纯中文前端会删除英文字符。规范化后，文本进入 `jieba.posseg` 分词、`pypinyin` 转拼音、变调处理和 `OpenCPOP` 音素表映射。

英文脚本 `english.py` 的正则化入口为：

```
1 def text_normalize(text):
2     text = text.lower()
3     text = expand_time_english(text)
4     text = normalize_numbers(text)
5     text = expand_abbreviations(text)
6     return text
7
```

它先将文本统一为小写，再展开时间、数字和常见缩写。例如时间和数字不能按字符逐个读出，而要转为自然英文读法；`Dr.`、`Mr.` 等缩写也需要展开。英文 G2P 阶段会优先查 CMU 词典，若词典中没有对应单词，则调用 `g2p_en.G2p()` 预测发音，并通过 `post_replace_ph` 处理未知符号和标点。

中英混合脚本 `chinese_mix.py` 的 `text_normalize` 与中文类似，也会进行数字转换和标点归一化，但关键区别是它保留英文字符、下划线和空格：

```
1 re.sub(r"[^\u4e00-\u9fa5_a-zA-Z\s" + "".join(punctuation) + r"]+", "", replaced_text)
2 re.sub(r"[\s]+", " ", replaced_text)
3
```

因此中英混合前端不会删除英文片段。之后它会按英文片段切分文本，如果某段匹配英文，就调用英文 G2P；否则调用中文 G2P。英文 tone 会加上英文声调空间的偏移，以便模型区分中文声调和英文重音。最终把中文和英文片段的 `phones`、`tones`、`word2ph` 拼接为统一序列。

三者差异总结如表 1 所示。

表 1: MeloTTS 中文、英文和中英混合文本前端对比

脚本	主要输入	正则化重点	后续处理特点
<code>chinese.py</code>	纯中文	数字转中文、中文标点统一、过滤非中文字符	jieba 分词、pypinyin 转拼音、变调和 OpenC-POP 音素映射
<code>english.py</code>	纯英文	小写化、时间展开、数字展开、缩写展开	BERT tokenizer 分词，优先查 CMU 词典，否则用 g2p_en 预测
<code>chinese_mix.py</code>	中英混合	数字转中文、标点统一，同时保留英文和空格	先切分中英文片段，再分别调用中文和英文 G2P，最后合并

问题二 请根据郭沫若先生的这一种读法，自定义 `phones`、`tones`、`word2ph` 列表，来正确地合成这副对联的语音。

答 原句为“海水朝朝朝朝朝朝朝落，浮云长长长长长长消。”，郭沫若先生给出的读法为：“海水潮，朝朝潮，朝潮朝落；浮云涨，长长涨，长涨长消”。也就是说，上联中“朝”需要在 `chao2` 和 `zhao1` 之间切换；下联中“长”需要在 `zhang3` 和 `chang2` 之间切换。默认文本前端会把多个“朝”和“长”读成同一种发音，因此需要手动指定音素、声调和文本到音素的对齐关系。

完成代码如下，生成音频文件为 `homework_2.wav`。

```
1 # 自定义两个子句的 phones、tones、word2ph 列表
2 phones1 = [
3     "_",
4     "h", "ai", "sh", "ui", "ch", "ao", ",",
5     "zh", "ao", "zh", "ao", "ch", "ao", ",",
6     "zh", "ao", "ch", "ao", "zh", "ao", "l", "uo", ".",
7     "_",
8 ]
9 tones1 = [
10     0,
11     3, 3, 3, 3, 2, 2, 0,
12     1, 1, 1, 1, 2, 2, 0,
13     1, 1, 2, 2, 1, 1, 4, 4, 0,
14     0,
```

```

15 ]
16 word2ph1 = [1, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 1, 1]
17
18 phones2 = [
19     "_",
20     "f", "u", "y", "vn", "zh", "ang", ",",
21     "ch", "ang", "ch", "ang", "zh", "ang", ",",
22     "ch", "ang", "zh", "ang", "ch", "ang", "x", "iao", ".",
23     "_",
24 ]
25 tones2 = [
26     0,
27     2, 2, 2, 2, 3, 3, 0,
28     2, 2, 2, 2, 3, 3, 0,
29     2, 2, 3, 3, 2, 2, 1, 1, 0,
30     0,
31 ]
32 word2ph2 = [1, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 2, 1, 1]
33
34 phones_customized = [phones1, phones2]
35 tones_customized = [tones1, tones2]
36 word2ph_customized = [word2ph1, word2ph2]
37
38 output_path = "homework_2.wav"
39 model.tts_to_file_custom_frontend(
40     text,
41     speaker_ids["ZH"],
42     output_path,
43     speed=speed,
44     phones_customized=phones_customized,
45     tones_customized=tones_customized,
46     word2ph_customized=word2ph_customized,
47 )
48

```

其中 word2ph 的作用是描述每个文本单元对应多少个音素位置。中文音节一般由声母和韵母两个音素组成，因此大部分汉字对应 2；句首、句尾和标点对应 1。代码中也加入了如下检查，保证音素、声调和对齐长度一致：

```

1 assert len(phones1) == len(tones1) == sum(word2ph1)
2 assert len(phones2) == len(tones2) == sum(word2ph2)
3

```

3 课程三：韵律控制（45%）

问题一 请介绍一下 VITS 模型的组成结构，训练和推理过程的工作原理。

答 VITS 是一种端到端文本到波形的语音合成模型。它不同于传统“声学模型预测 mel 频谱、声码器再生成波形”的两阶段方法，而是把文本编码、声学隐变量建模、对齐/时长建模和波形生成放在一个可联合训练的框架中。MeloTTS 的核心声学模型沿用了 VITS 类结构，并在多语言、多说话人、BERT 特征和文本前端方面做了扩展。

VITS 可以看作 CVAE、normalizing flow、duration/alignment 模块和 GAN vocoder 的组合。其主要结构如下：

1. 文本编码器 Text Encoder: 输入文本经过正则化和 G2P 后变成音素序列，并结合声调、语言 ID、说话人 ID 和 BERT 特征。文本编码器输出音素级隐藏表示，并预测文本先验分布参数 m_p 与 $\log s_p$ 。
2. 后验编码器 Posterior Encoder: 训练时真实语音已知，模型从真实语音谱特征中编码出语音侧隐变量 z ，得到后验分布参数 m_q 与 $\log s_q$ 。
3. Normalizing Flow: 用于连接文本先验空间和语音后验空间。训练时把后验隐变量映射到先验空间以计算似然和 KL 约束；推理时反向使用 flow，把从文本先验采样的隐变量变换到解码器空间。
4. 对齐和时长模块: 训练时使用 MAS 搜索文本音素和语音帧之间的最优单调对齐，得到每个音素的持续帧数；推理时使用 Duration Predictor 或 Stochastic Duration Predictor 根据文本条件预测音素时长。
5. 解码器 Decoder: 通常采用 HiFi-GAN 风格结构，直接把隐变量 z 解码成时域波形。
6. 判别器 Discriminators: 通过对抗训练提升波形自然度，常见结构包括多周期判别器和多尺度判别器。训练损失包括对抗损失、特征匹配损失、重建损失、KL 损失和时长损失等。

训练阶段同时输入文本和真实语音。文本编码器给出文本先验分布，后验编码器从真实语音得到后验隐变量，flow 将后验隐变量映射到文本先验空间。随后 MAS 在文本音素和语音帧之间搜索最大似然的单调对齐路径，并由该路径得到每个音素的时长。解码器根据后验隐变量生成波形，判别器判断真实语音与生成语音，从而共同优化生成器、判别器和时长预测模块。

推理阶段只有文本，没有真实语音，因此不再使用后验编码器，也无法运行 MAS。此时文本经过前端和文本编码器得到先验分布参数，SDP 预测每个音素的持续时间，生成对齐路径；模型再从文本先验采样隐变量，经 flow 反向变换后由解码器直接生成波形。MeloTTS 中的 `speed`、`sdp_ratio`、`noise_scale_w` 和本实验修改的 `w_ceil` 都是在影响推理阶段的时长和韵律。

问题二 请介绍一下 VITS 中的 Monotonic Alignment Search (MAS) 算法和 Stochastic Duration Predictor 模块，说明它们在时长建模和韵律生成中的作用。

答 MAS 和 SDP 都与时长建模有关，但二者工作阶段不同。MAS 主要用于训练阶段，从真实语音中找出文本音素和语音帧之间的最优对齐；SDP 主要用于推理阶段，根据文本条件预测音素持续时间，并通过随机建模生成更自然的韵律变化。

MAS 的全称是 Monotonic Alignment Search，即单调对齐搜索。TTS 中文本序列和语音帧序列长度不同，一句话可能只有几十个音素，但对应几百帧语音。合理的对齐应满足单调性和连续性：语音按文本顺序发出，后面的音素不能对齐到前面语音帧，一个音素通常对应连续若干帧。训练时，后验编码器从真实语音得到隐变量 z ，flow 将其映射到先验空间得到 z_p ；文本编码器为每个音素预测先

验分布参数 $m_p, \log s_p$ 。由此可以计算每个语音帧在每个音素先验分布下的似然，MAS 在这个得分矩阵中寻找满足单调约束且总似然最大的路径。

MeloTTS/VITS 训练代码中的核心逻辑可概括为：

```
1 attn = monotonic_align.maximum_path(neg_cent, attn_mask.squeeze(1))
2 w = attn.sum(2)
3
```

其中 `attn` 是文本音素到语音帧的硬对齐矩阵，`w` 是每个音素对应的帧数。MAS 的作用是自动从训练数据中发现音素时长监督信号，并为时长预测器提供训练目标。

SDP 的全称是 Stochastic Duration Predictor，即随机时长预测器。它根据文本编码器输出的音素级表示预测每个音素持续多久。普通确定性时长预测器只能给出一个固定时长，而真实说话中同一句话可以有不同节奏、停顿和拖音。SDP 使用概率模型建模这种一对多关系，使推理时可以从时长分布中采样，从而提高韵律自然度。MeloTTS 推理中与时长相关的代码可概括为：

```
1 logw = self.sdp(..., reverse=True, noise_scale=noise_scale_w) * sdp_ratio + ...
2 w = torch.exp(logw) * x_mask * length_scale
3 w_ceil = torch.ceil(w)
4 attn = commons.generate_path(w_ceil, attn_mask)
5
```

其中 `w_ceil` 直接表示每个音素持续的帧数。它越大，音素持续越久，语音越慢；越小，音素越短，语速越快。训练时 MAS 提供真实时长，SDP 学习条件时长分布；推理时 SDP 预测时长并生成对齐路径。因此 MAS 解决“训练时如何对齐”，SDP 解决“推理时如何决定时长”，二者共同决定 VITS 的韵律建模能力。

问题三 请通过调整 `w_ceil` 来控制合成语音的韵律，合成一句“你听到了吗？这句话我会说得越来越慢。”的语音。要求语速越来越慢：句首接近原始时长、句尾约为原始时长的 3 倍（对应语速约从 1 倍线性降至 0.33 倍）。提交代码和生成的音频文件（命名为 `homework_3.wav`）。

答 本实验先调用扩展后的接口获取模型原始预测的 `w_ceil`，然后按照音素在整句中的全局位置线性放大其时长。第一个音素的缩放系数接近 1，最后一个音素的缩放系数接近 3，从而实现从正常语速逐步变慢到约三倍时长的效果。核心代码如下，最终生成文件为 `homework_3.wav`。

```
1 from melo.api import TTS
2 from IPython.display import Audio
3 import torch
4
5 text = "你听到了吗？这句话我会说得越来越慢。"
6 speed = 1.0
7 device = "cuda:0" if torch.cuda.is_available() else "cpu"
8 output_path = "homework_3.wav"
9
10 model = TTS(language="ZH", device=device)
11 speaker_ids = model.hps.data.spk2id
12
13 original_w_ceil_list, phone_list, tone_list = model.get_original_w_ceil(
```

```

14     text,
15     speaker_ids["ZH"],
16     speed=speed,
17     sdp_ratio=0,
18     noise_scale=0,
19     noise_scale_w=0,
20     quiet=True,
21 )
22
23 original_duration_lists = [
24     w.squeeze().detach().cpu().int().tolist()
25     for w in original_w_ceil_list
26 ]
27 total_phone_count = sum(len(duration_list) for duration_list in original_duration_lists)
28
29 modified_w_ceil_list = []
30 global_index = 0
31 for duration_list in original_duration_lists:
32     modified_duration_list = []
33     for original_duration in duration_list:
34         if total_phone_count <= 1:
35             scale = 1.0
36         else:
37             scale = 1.0 + 2.0 * global_index / (total_phone_count - 1)
38             modified_duration = max(1, int(round(original_duration * scale)))
39             modified_duration_list.append(modified_duration)
40             global_index += 1
41     modified_w_ceil_list.append(modified_duration_list)
42
43 print("original w_ceil:", original_duration_lists)
44 print("modified w_ceil:", modified_w_ceil_list)
45
46 model.tts_to_file_custom_duration(
47     text,
48     speaker_ids["ZH"],
49     output_path,
50     speed=speed,
51     sdp_ratio=0,
52     noise_scale=0,
53     noise_scale_w=0,
54     quiet=True,
55     w_ceil_customized=modified_w_ceil_list,
56 )
57
58 Audio(output_path)
59

```

实验听感上，句首“你听到了吗”接近正常速度，后半句“越来越慢”明显拖长，整体语速呈逐渐下降趋势，说明直接修改 `w_ceil` 可以细粒度控制 VITS 推理阶段的韵律。

4 课程四：MeloTTS 的训练与推理优化（加分项 10%）

4.1 所选选题：选题一

MeloTTS 的说话人音色训练 请参考 MeloTTS 的官方训练教程，从原神语音数据中任选一个角色语音作为数据集，训练一个新的中文说话人音色 MeloTTS 模型。并从数据集中至少划分 30 条语音作为测试集，评测 CER 和说话人相似度。

要求：

- 1. 说明选择的说话人名称、训练集时长，时长 ≥ 10 分钟；
- 2. 简要说明训练过程，包含 loss 变化截图；
- 3. 提供测试集的 CER 和说话人相似度评测结果。

4.2 实现过程

本实验选择原神角色娜维娅（Navia）作为目标说话人。数据集位于 Navia/，包含 1614 条 wav/lab 配对数据，总时长约 140.03 分钟，原始采样率为 48000 Hz，单声道，满足实验要求中训练集时长不少于 10 分钟的条件。实验从中划分 30 条作为测试集，其余作为训练和验证数据；预处理时再从训练数据中划分 8 条验证集。

实验环境为 Windows 下的 melotts conda 环境，使用 GPU 版 PyTorch。由于题目提示官方英文预训练模型直接用于中文可能效果较差，本实验严格采用题目推荐的中文预训练模型 CJY1018/MeloTTS-ZH-BZNSYP/，加载其中的 G_325000.pth、D_325000.pth 和 DUR_325000.pth 进行微调。中文 BERT 模型 chinese-roberta-wwm-ext 采用手动下载到本地目录的方式加载，避免训练过程中反复访问 Hugging Face。

数据准备阶段主要完成三件事：第一，读取 Navia/ 中的 wav/lab 文件；第二，将 lab 文本开头的 # 去掉，并将 {NICKNAME} 替换为“旅行者”；第三，将原始 48000 Hz 音频统一重采样为 44100 Hz 单声道，保存到 melo/data/navia/wavs_44100/。关键代码如下：

```
1 REPO_ROOT = Path(r"D:\语音信息处理\labcode\实验4\MeloTTS-Homework")
2 MELO_DIR = REPO_ROOT / "melo"
3 NAVIA_RAW_DIR = REPO_ROOT / "Navia"
4 WORK_DIR = MELO_DIR / "data" / "navia"
5 WAV_DIR = WORK_DIR / "wavs_44100"
6 METADATA_PATH = WORK_DIR / "metadata.list"
7 TEST_LIST_PATH = WORK_DIR / "test.list"
8
9 def clean_lab_text(text: str) -> str:
10     text = text.strip()
11     if text.startswith("#"):
12         text = text[1:].strip()
13     text = text.replace("{NICKNAME}", "旅行者")
14     return text
15
16 # 将原始 wav 统一转为 44100 Hz 单声道，并生成 metadata.list/test.list
17 metadata_lines = []
18 test_lines = []
```

```

19 for idx, lab_path in enumerate(sorted(NAVIA_RAW_DIR.glob("*.lab")), 1):
20     wav_path = lab_path.with_suffix(".wav")
21     text = clean_lab_text(lab_path.read_text(encoding="utf-8"))
22     out_wav = WAV_DIR / wav_path.name
23     # 实际 notebook 中使用 torchaudio 完成重采样和保存
24     rel_path = out_wav.relative_to(MELO_DIR).as_posix()
25     line = f"{rel_path}|navia|ZH|{text}"
26     if len(test_lines) < 30:
27         test_lines.append(line)
28     else:
29         metadata_lines.append(line)
30
31 METADATA_PATH.write_text("\n".join(metadata_lines) + "\n", encoding="utf-8")
32 TEST_LIST_PATH.write_text("\n".join(test_lines) + "\n", encoding="utf-8")
33

```

文本预处理阶段对 `metadata.list` 中的每条文本执行中文文本正则化、G2P 和 BERT 特征生成，保存 `metadata.list.cleaned`、`train.list`、`val.list` 以及每条语音对应的 `.bert.pt`。同时以中文预训练模型的 `config.json` 为基础，修改说话人映射、训练列表、验证列表、采样率和训练参数。关键代码如下：

```

1 PRETRAIN_DIR = REPO_ROOT / "CJY1018" / "MeLoTTS-ZH-BZNSYP"
2 PRETRAIN_CONFIG = PRETRAIN_DIR / "config.json"
3 BERT_MODEL_DIR = REPO_ROOT / "chinese-roberta-wwm-ext-large"
4 os.environ["CHINESE_ROBERTA_WWM_EXT_LARGE"] = str(BERT_MODEL_DIR)
5
6 from text.chinese_bert import get_bert_feature as get_zh_bert_feature
7 import text.chinese as chinese
8
9 def clean_text_bert_zh(text, language, device=None):
10     norm_text = chinese.text_normalize(text)
11     phones, tones, word2ph = chinese.g2p(norm_text)
12     word2ph_for_bert = copy.deepcopy(word2ph)
13     for i in range(len(word2ph_for_bert)):
14         word2ph_for_bert[i] = word2ph_for_bert[i] * 2
15     word2ph_for_bert[0] += 1
16     bert = get_zh_bert_feature(norm_text, word2ph_for_bert, device=device)
17     return norm_text, phones, tones, word2ph, bert
18
19 for line in metadata.read_text(encoding="utf-8").splitlines():
20     utt, spk, language, text = line.strip().split("|", 3)
21     norm_text, phones, tones, word2ph, bert = clean_text_bert_zh(text, language, device="cuda:0")
22     bert_path = (MELO_DIR / utt).with_suffix(".bert.pt")
23     torch.save(bert.cpu(), bert_path)
24

```

训练阶段在 Windows/Jupyter 环境中直接在当前内核调用 `train.run()`，并设置单卡分布式相关环境变量。为避免 Windows 下子进程加载 `_ctypes` 和 `DataLoader worker` 派生问题，训练时将

`num_workers` 改为 0, 并设置 `USE_LIBUV=0`。关键训练参数如下:

```

1 RUN_TRAINING = True
2 train_args = [
3     "train.py",
4     "--config", "data/navia/config.json",
5     "--model", "navia",
6     "--pretrain_G", str(PRETRAIN_G),
7     "--pretrain_D", str(PRETRAIN_D),
8     "--pretrain_dur", str(PRETRAIN_DUR),
9 ]
10
11 os.environ["LOCAL_RANK"] = "0"
12 os.environ["RANK"] = "0"
13 os.environ["WORLD_SIZE"] = "1"
14 os.environ["USE_LIBUV"] = "0"
15 os.environ.setdefault("MASTER_ADDR", "127.0.0.1")
16 os.environ.setdefault("MASTER_PORT", "29501")
17
18 sys.argv = train_args
19 import train as train_module
20 train_module.run()
21

```

训练日志和 checkpoint 保存到 `melo/logs/navia/`。本次训练保存到约 7500 step, 可见 `G_7500.pth`、`D_7500.pth` 和 `DUR_7500.pth`。TensorBoard 中观察到 `loss/g/total` 和 `loss/d/total` 均有明显波动, 而非单调下降。这是 VITS/MeloTTS 中生成器和判别器对抗训练的正常现象: 生成器和判别器互相博弈, 二者 loss 会随对方能力变化而上下震荡, 最低 loss 也不一定对应主观听感最好的 checkpoint。图 4.1 和图 4.2 展示了训练过程中的 loss 曲线。

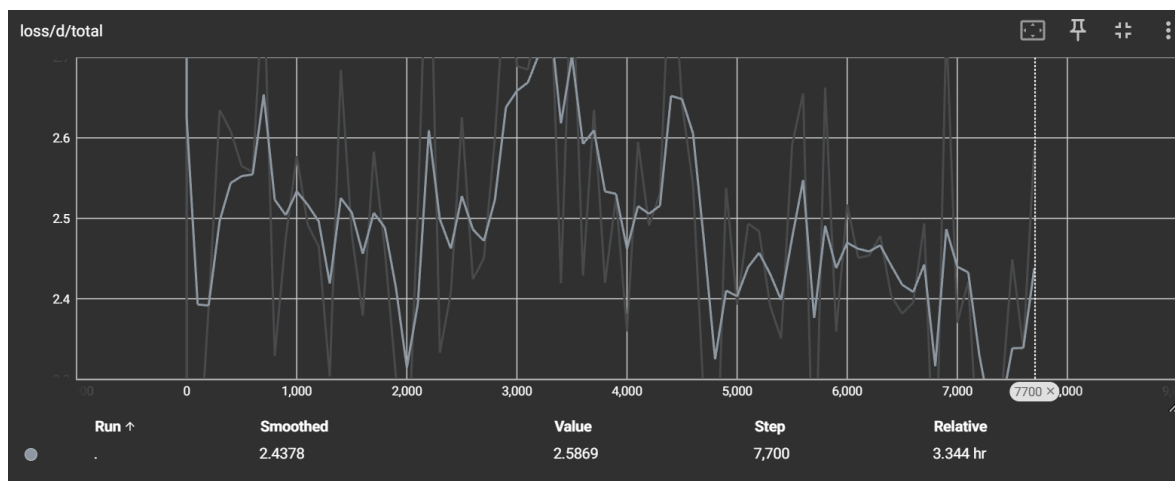


图 4.1: 判别器总损失 `loss/d/total` 曲线

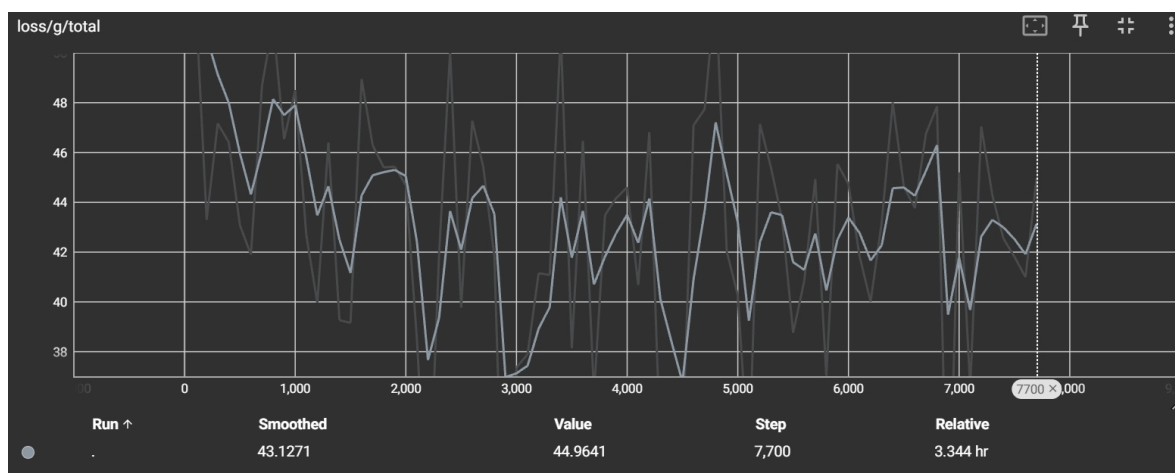


图 4.2: 生成器总损失 loss/g/total 曲线

训练后选择最新的 G_7500.pth 对测试集 30 条文本进行合成,输出目录为 melo/outputs/navia_test/。核心推理代码如下:

```
1 from melo.api import TTS
2
3 CKPT_PATH = LOG_DIR / "G_7500.pth"
4 OUTPUT_TEST_DIR = MELO_DIR / "outputs" / "navia_test"
5 OUTPUT_TEST_DIR.mkdir(parents=True, exist_ok=True)
6
7 tts_model = TTS(
8     language="ZH",
9     device="cuda:0",
10    config_path=str(LOG_DIR / "config.json"),
11    ckpt_path=str(CKPT_PATH),
12 )
13 speaker_id = tts_model.hps.data.spk2id["navia"]
14
15 for idx, line in enumerate(TEST_LIST_PATH.read_text(encoding="utf-8").splitlines(), 1):
16     wav_rel, spk, lang, text = line.split("|", 3)
17     out_wav = OUTPUT_TEST_DIR / f"navia_gen_{idx:03d}.wav"
18     tts_model.tts_to_file(text, speaker_id, str(out_wav), speed=1.0, quiet=True)
19
```

4.3 评测结果

CER 使用 Whisper 对 30 条合成音频做中文 ASR, 再与 test.list 中参考文本计算字符级编辑距离。计算公式为:

$$CER = \frac{\text{EditDistance}(\text{ref}, \text{hyp})}{|\text{ref}|}$$

核心代码如下:

```
1 def normalize_for_cer(text: str) -> str:
2     pattern = r"[\s, . ! ? , ; : , . ! ? ; \ ' " " ' ( ) 《 》 【 】 \[ \] — … - ]"""
```

```

3     text = re.sub(pattern, "", text)
4     return text.lower()
5
6 def edit_distance(a: str, b: str) -> int:
7     dp = list(range(len(b) + 1))
8     for i, ca in enumerate(a, 1):
9         prev, dp[0] = dp[0], i
10        for j, cb in enumerate(b, 1):
11            old = dp[j]
12            dp[j] = min(dp[j] + 1, dp[j - 1] + 1, prev + (ca != cb))
13            prev = old
14    return dp[-1]
15
16 asr_model = whisper.load_model("base", device=device, download_root=str(WHISPER_MODEL_DIR))
17 total_edits = 0
18 total_chars = 0
19 for idx, line in enumerate(TEST_LIST_PATH.read_text(encoding="utf-8").splitlines(), 1):
20     ref_wav_rel, spk, lang, ref_text = line.split("|", 3)
21     gen_wav = OUTPUT_TEST_DIR / f"navia_gen_{idx:03d}.wav"
22     result = asr_model.transcribe(str(gen_wav), language="zh", fp16=torch.cuda.is_available())
23     hyp_text = result["text"].strip()
24     ref_norm = normalize_for_cer(ref_text)
25     hyp_norm = normalize_for_cer(hyp_text)
26     total_edits += edit_distance(ref_norm, hyp_norm)
27     total_chars += len(ref_norm)
28 cer_avg = total_edits / max(1, total_chars)
29

```

说话人相似度使用 microsoft/wavlm-base-plus-sv 提取真实测试语音和对应合成语音的 speaker embedding，并计算余弦相似度。核心代码如下：

```

1 feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained(
2     str(WAVLM_MODEL_DIR), local_files_only=True
3 )
4 sv_model = WavLMForXVector.from_pretrained(
5     str(WAVLM_MODEL_DIR), local_files_only=True
6 ).to(device)
7 sv_model.eval()
8
9 def speaker_embedding(path: Path):
10     audio = load_16k(path)
11     inputs = feature_extractor(audio, sampling_rate=16000, return_tensors="pt", padding=True)
12     inputs = {k: v.to(device) for k, v in inputs.items()}
13     with torch.no_grad():
14         emb = sv_model(**inputs).embeddings
15         emb = F.normalize(emb, dim=-1)
16     return emb.squeeze(0).cpu()
17
18 sims = []

```

```
19 for idx, line in enumerate(TEST_LIST_PATH.read_text(encoding="utf-8").splitlines(), 1):
20     ref_wav_rel, spk, lang, ref_text = line.split("|", 3)
21     ref_wav = MEL0_DIR / ref_wav_rel
22     gen_wav = OUTPUT_TEST_DIR / f"navia_gen_{idx:03d}.wav"
23     sim = F.cosine_similarity(speaker_embedding(ref_wav), speaker_embedding(gen_wav),
24                             dim=0).item()
25     sims.append(sim)
26 mean_similarity = sum(sims) / len(sims)
```

本次评测输出如下：

```
1 平均 CER: 0.3277027027027027
2 平均说话人相似度: 0.8637855644027392
3
```

结果分析如下：平均说话人相似度为 0.864，说明模型较好地学习到了娜维娅的音色特征，与主观听感中“音色接近目标角色”的观察一致。平均 CER 为 0.328，说明合成语音整体可识别，但可懂度仍有提升空间，可能存在发音不稳、漏字或 Whisper 对角色腔调识别误差等问题。

从主观听感来看，微调模型较明显地学习到了目标说话人的音色，但语气、语调和韵律仍偏机械。这与 MeloTTS 的训练方式有关：本实验使用的是文本-语音配对微调，没有引入情感标签、风格 token 或参考音频风格编码；同时原神角色语音多为短句和场景化台词，情绪变化丰富但缺乏显式监督。因此模型更容易学习稳定的说话人 timbre，而较难学习高层次的 prosody 和 emotion。后续若要改善韵律，可尝试筛选风格一致的数据子集、增加训练步数、降低学习率继续微调，或引入带情感/风格控制能力的 TTS 模型。